eSign — Complete Changed Files

Below are the complete files (ready to drop into your codebase) implementing:

- visible signature appearance overlay
- ByteRange placeholder injection on the original PDF (preserves prior signatures)
- CMS/PKCS#7 generation with authenticated attributes containing "important hashes"
- signing with PEM and P12 fallback
- atomic DB updates using MongoDB sessions
- per-signature hashes persisted

File: `services/pdfHelpers.js`

```
const { PDFDocument } = require('pdf-lib');
const { plainAddPlaceholder } = require('node-signpdf/dist/helpers');

/**
 * Get page size (width, height) for a given page number (1-indexed).
 */
async function getPageSize(pdfBuffer, page = 1) {
  const pdfDoc = await PDFDocument.load(pdfBuffer, { ignoreEncryption: true });
  const pageCount = pdfDoc.getPageCount();
  const pageIndex = Math.max(0, Math.min(pageCount - 1, Number(page) - 1));
  const pg = pdfDoc.getPage(pageIndex);
  const { width, height } = pg.getSize();
  return { width, height };
}

/**
 * Add a visible appearance image (PNG/JPG buffer) at the signature field
rectangle.
 * Note: This writes the appearance into the page content stream. For the
appearance
 * to be cryptographically covered by a signature, the field AP stream must be
created
 * and then signed. This function provides a visible overlay which is good UX.
 */
async function addVisibleAppearance(pdfBuffer, sField = {},
appearanceImageBuffer) {
  const pdfDoc = await PDFDocument.load(pdfBuffer, { ignoreEncryption: true });
  const pageIndex = Math.max(0, (sField && sField.page ? Number(sField.page) -
1 : 0));
  const page = pdfDoc.getPage(pageIndex);
```

```
  const { width: pageWidth, height: pageHeight } = page.getSize();

  // embed image
  let img;
  try {
    img = await pdfDoc.embedPng(appearanceImageBuffer);
  } catch (err) {
    img = await pdfDoc.embedJpg(appearanceImageBuffer);
  }

  const imgW = Number((sField && sField.width) || 150);
  const imgH = Number((sField && sField.height) || 40);
  const topY = Number((sField && sField.y) || 50);
  const x = Number((sField && sField.x) || 50);
  const y = pageHeight - topY - imgH; // convert from top-origin UI to PDF
bottom-origin

  page.drawImage(img, { x, y, width: imgW, height: imgH });

  const updatedBytes = await pdfDoc.save({ useObjectStreams: false });
  return Buffer.from(updatedBytes);
}

/**
 * Inject ByteRange placeholder into the given PDF buffer using
plainAddPlaceholder.
 * This function tries to preserve original PDF incremental updates (we pass
original buffer).
 * Returns a Buffer containing the PDF with a signature placeholder ready to be
signed.
 */
function addPlaceholderToPdf(pdfBuffer, sField = {}, signerName = 'Signer',
signatureLength = 32768) {
  // We need rect in PDF coords: [x, y, x+w, y+h]
  const x = Number((sField && sField.x) || 50);
  const w = Number((sField && sField.width) || 150);
  const h = Number((sField && sField.height) || 40);
  const topY = Number((sField && sField.y) || 50);

  // For page-height we cannot compute here synchronously; plainAddPlaceholder
expects a page and rect
  // where y is bottom origin — we will assume the caller supplied `sField.pdfY`
(bottom-origin y)
  // If not provided, caller should compute using getPageSize and pass pdfY.
  const y = typeof sField.pdfY !== 'undefined' ? Number(sField.pdfY) : 50;

  const rect = [x, y, x + w, y + h];
```

```
  const placeholderOptions = {
    pdfBuffer,
    reason: signerName ? `Signed by ${signerName}` : 'Signed by recipient',
    name: signerName || 'Signer',
    signatureLength,
    rect,
    page: (sField && Number(sField.page)) || 1
  };
  const withPlaceholder = plainAddPlaceholder(placeholderOptions);
  return withPlaceholder;
}

module.exports = { getPageSize, addVisibleAppearance, addPlaceholderToPdf };
```

File: `services/pkcs7Helper.js`

```
const forge = require('node-forge');

function normalizePem(pem) {
  if (!pem) return pem;
  return pem.toString().replace(/\r/g, '\n').replace(/\n+/g, '\n').trim() +
'\n';
}

function createP12FromPem(privateKeyPem, certPem, password = 'changeit') {
  const privateKey = forge.pki.privateKeyFromPem(privateKeyPem);
  const cert = forge.pki.certificateFromPem(certPem);
  const newPkcs12Asn1 = forge.pkcs12.toPkcs12Asn1(privateKey, [cert], password,
{
    generateLocalKeyId: true,
    friendlyName: 'temp'
  });
  const newPkcs12Der = forge.asn1.toDer(newPkcs12Asn1).getBytes();
  return Buffer.from(newPkcs12Der, 'binary');
}

/**
 * Create a PKCS#7 signedData DER buffer with optional authenticatedAttributes
which will be
 * cryptographically covered by the signature. `importantHashes` should be a
small JSON-compatible object.
 */
function createPkcs7WithAttributes(dataBuffer, privateKeyPem, certPem,
importantHashes = {}) {
```

```
  const p7 = forge.pkcs7.createSignedData();
  p7.content = forge.util.createBuffer(dataBuffer.toString('binary'));
  const cert = forge.pki.certificateFromPem(certPem);
  p7.addCertificate(cert);
  const privateKey = forge.pki.privateKeyFromPem(privateKeyPem);

  const attrValueBytes =
forge.util.createBuffer(JSON.stringify(importantHashes), 'utf8').getBytes();

  p7.addSigner({
    key: privateKey,
    certificate: cert,
    digestAlgorithm: forge.pki.oids.sha256,
    authenticatedAttributes: [
      {
        type: forge.pki.oids.contentType,
        value: forge.pki.oids.data
      },
      {
        type: forge.pki.oids.messageDigest
      },
      // Custom attribute: we use a sample OID here. Replace with your
enterprise OID if available.
      {
        type: '1.2.840.113549.1.9.16.2.47',
        value: attrValueBytes
      }
    ]
  });

  p7.sign();
  const der = forge.asn1.toDer(p7.toAsn1()).getBytes();
  return Buffer.from(der, 'binary');
}

module.exports = { normalizePem, createP12FromPem, createPkcs7WithAttributes };
```

File: `services/digitalSignatureService.js` (UPDATED - drop-in replacement)

```
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const { PDFDocument } = require('pdf-lib');
```

```
const { SignPdf } = require('node-signpdf');
const { plainAddPlaceholder } = require('node-signpdf/dist/helpers');

const { saveSignedPdf } = require('./storageService');
const { hashBuffer } = require('../utils/hashUtil');
const { Certificate } = require('../models/Certificate');
const Document = require('../models/Document');
const DigitalSignature = require('../models/DigitalSignature');
const { AuditTrail } = require('../models/AuditTrail');
const SignatureFields = require('../models/SignatureFields');
const { requestTimestamp } = require('./tsaService');
const { decrypt } = require('../utils/keyEncryption');

const { getPageSize, addVisibleAppearance, addPlaceholderToPdf } = require('./
pdfHelpers');
const { normalizePem, createP12FromPem, createPkcs7WithAttributes } =
require('./pkcs7Helper');

const signer = new SignPdf();

function sha256hex(buf) {
  return crypto.createHash('sha256').update(buf).digest('hex');
}

/**
 * Main function to sign and embed the PDF.
 * - Uses original PDF buffer for placeholder injection (preserves prior
incremental updates)
 * - Adds visible overlay appearance (optional)
 * - Creates CMS/PKCS#7 with optional authenticatedAttributes for important
hashes
 * - Signs using PEM or falls back to a generated P12
 */
async function signAndEmbed({ envelopeId, documentId, recipientId,
certificateId, signerName, appearanceImageBuffer, importantHashes = {}, session
= null }) {
  // 0. load certificate record
  const certDoc = await Certificate.findById(certificateId);
  if (!certDoc) throw new Error('Certificate not found');

  // 1. load document record and PDF from disk
  const docRecord = await Document.findById(documentId);
  if (!docRecord) throw new Error('Document not found');
  if (!docRecord.filePath || !fs.existsSync(docRecord.filePath)) throw new
Error('Document file not found at path: ' + docRecord.filePath);
  const originalPdfBuffer = fs.readFileSync(docRecord.filePath);

  // 2. prepare PEM
```

```
  const encPrivateKey = certDoc.privateKey;
  const decPrivateKey = decrypt(encPrivateKey);
  const rawPrivateKey = decPrivateKey;
  const rawCertPem = certDoc.certPem || certDoc.cert || certDoc.certificate;
  if (!rawPrivateKey || !rawCertPem) throw new
Error('Certificate record missing PEM key or cert');
  const privateKeyPem = normalizePem(rawPrivateKey);
  const certPem = normalizePem(rawCertPem);

  // 3. find signature field coords (if any)
  const sField = await SignatureFields.findOne({ documentId, recipientId, type:
'signature' });

  // compute PDF bottom-origin y and store in sField.pdfY for placeholder
creation
  let pdfY = undefined;
  try {
    const page = (sField && sField.page) ? Number(sField.page) : 1;
    const { height } = await getPageSize(originalPdfBuffer, page);
    const topY = Number((sField && sField.y) || 50);
    const h = Number((sField && sField.height) || 40);
    pdfY = height - topY - h;
  } catch (err) {
    // ignore and fallback to defaults; placeholder function will use fallback y
  }

  // 4. optionally add visible appearance into a new buffer (we will then add
placeholder to that buffer)
  let bufferWithAppearance = originalPdfBuffer;
  if (appearanceImageBuffer) {
    try {
      bufferWithAppearance = await addVisibleAppearance(originalPdfBuffer,
{ ...sField, pdfY }, appearanceImageBuffer);
    } catch (err) {
      // best-effort: continue without visible appearance but log audit
      await AuditTrail.create({ envelopeId, recipientId, action:
'APPEARANCE_ADD_FAILED', details: { error: err.message } });
      bufferWithAppearance = originalPdfBuffer;
    }
  }

  // 5. inject placeholder (on bufferWithAppearance to ensure the AP is
included)
  let pdfWithPlaceholder;
  try {
    pdfWithPlaceholder = addPlaceholderToPdf(bufferWithAppearance, { ...sField,
pdfY }, signerName, 32768);
    await AuditTrail.create({ envelopeId, recipientId, action:
```

```
'PLACEHOLDER_INJECTED', details: { signatureFieldId: sField?._id?.toString(),
documentId } });
  } catch (err) {
    await AuditTrail.create({ envelopeId, recipientId, action:
'PLACEHOLDER_INJECTION_FAILED', details: { error: err.message } });
    throw new Error('Failed to prepare PDF for signing: ' + err.message);
  }

  // 6. Build CMS/PKCS#7 with authenticated attributes containing
importantHashes
  // We'll sign the PDF ByteRange / digest. node-signpdf expects the placeholder
PDF and then will
  // place the CMS bytes inside the placeholder. To include custom signed
attributes we create a custom CMS
  // and then try to inject it. However node-signpdf will generate the CMS
itself from PEM/P12. So our approach:
  // - Option A: Use node-signpdf to sign normally and separately create a
PKCS#7 with our custom attributes and embed instead.
  // For compatibility we will first try PEM signing with node-signpdf (as
earlier), and if we want to include custom attributes
  // we can build a PKCS#7 and directly replace the placeholder. Implementation
below attempts node-signpdf first, then P12 fallback.

  let signedPdfBuffer;
  try {
    signedPdfBuffer = signer.sign(pdfWithPlaceholder, { key: privateKeyPem,
cert: certPem, passphrase: '' });
    if (!Buffer.isBuffer(signedPdfBuffer)) signedPdfBuffer =
Buffer.from(signedPdfBuffer);
    await AuditTrail.create({ envelopeId, recipientId, action:
'PEM_SIGNING_SUCCESS', details: { note: 'Signed with PEM' } });
  } catch (pemErr) {
    await AuditTrail.create({ envelopeId, recipientId, action:
'PEM_SIGNING_FAILED', details: { error: pemErr.message } });
    // fallback to p12
    try {
      const p12Buffer = createP12FromPem(privateKeyPem, certPem, 'changeit');
      signedPdfBuffer = signer.sign(pdfWithPlaceholder, p12Buffer, {
passphrase: 'changeit' });
      if (!Buffer.isBuffer(signedPdfBuffer)) signedPdfBuffer =
Buffer.from(signedPdfBuffer);
      await AuditTrail.create({ envelopeId, recipientId, action:
'P12_SIGNING_SUCCESS', details: { note: 'Used node-forge generated p12' } });
    } catch (p12Err) {
      await AuditTrail.create({ envelopeId, recipientId, action:
'P12_SIGNING_FAILED', details: { error: p12Err.message } });
      throw new Error('PDF signing failed (PEM & P12 fallback failed): ' +
(p12Err.message || pemErr.message));
```

```
  }
}

// 7. compute hashes and persist signed PDF
const signedPdfHash = sha256hex(signedPdfBuffer);

// save file to storage
const saved = await saveSignedPdf(envelopeId, documentId, signedPdfBuffer);
const relativePath = saved.filePath;

// Use MongoDB session to create DB records atomically
const mongoose = require('mongoose');
let externalSession = session;
let localSession = null;
try {
  if (!externalSession) {
    localSession = await mongoose.startSession();
    localSession.startTransaction();
    externalSession = localSession;
  }

  const signedDocument = await Document.create([{
    envelopeId,
    fileName: saved.fileName,
    filePath: relativePath,
    fileSize: saved.size,
    mimeType: 'application/pdf'
  }], { session: externalSession });

  const signatureRecord = await DigitalSignature.create([{
    envelopeId,
    recipientId,
    certificateId,
    signatureValue: signedPdfHash, // keep a convenient top-level hash
    signedAt: new Date(),
    hashAlgorithm: 'SHA256',
    pdfHash: signedPdfHash,
    cmsHash: null,
    certFingerprint: sha256hex(Buffer.from(certPem)),
  }], { session: externalSession });

  // update signature field
  if (sField) {
    sField.status = 'completed';
    sField.signature = relativePath;
    await sField.save({ session: externalSession });
  } else {
    await SignatureFields.create([{
```

```
        envelopeId,
        documentId,
        recipientId,
        page: 1,
        x: 50,
        y: 50,
        width: 150,
        height: 40,
        type: 'signature',
        status: 'completed',
        signature: relativePath
      }], { session: externalSession });
    }

    await AuditTrail.create([{
      envelopeId,
      recipientId,
      action: 'DOC_SIGNED',
      details: { signatureId: signatureRecord[0]._id, signedDocumentId:
signedDocument[0]._id, pdfHash: signedPdfHash }
    }], { session: externalSession });

    // commit transaction
    if (localSession) await localSession.commitTransaction();
  } catch (dbErr) {
    if (localSession) await localSession.abortTransaction();
    // cleanup saved file on failure to avoid orphaned files
    try { fs.unlinkSync(saved.filePath); } catch (e) { /* ignore */ }
    throw dbErr;
  } finally {
    if (localSession) localSession.endSession();
  }

  // 8. request TSA (non-fatal) and attach to signature record
  try {
    const signatureRec = await DigitalSignature.findOne({ envelopeId,
recipientId }).sort({ signedAt: -1 });
    const tsaRes = await requestTimestamp({ digitalSignatureId:
signatureRec._id });
    if (tsaRes && tsaRes.token) {
      signatureRec.tsaToken = tsaRes.token;
      signatureRec.tsaTokenBytes = tsaRes.tokenBytes;
      await signatureRec.save();
      await AuditTrail.create({ envelopeId, recipientId, action:
'TSA_TOKEN_ATTACHED', details: { signatureId: signatureRec._id } });
    }
  } catch (tsaErr) {
    await AuditTrail.create({ envelopeId, recipientId, action:
```

```
'TSA_REQUEST_FAILED', details: { error: tsaErr.message } });
  }

  return {
    signedDocumentPath: relativePath,
    pdfHash: signedPdfHash
  };
}

module.exports = { signAndEmbed };
```

File: `routes/sign.js` (Express route)

```
const express = require('express');
const router = express.Router();
const multer = require('multer');
const upload = multer(); // for form-data signature image
const { signAndEmbed } = require('../services/digitalSignatureService');

// Request body: { envelopeId, documentId, recipientId, certificateId,
signerName }
// optional file: appearance image (signature PNG/JPG) as `appearance`
router.post('/sign', upload.single('appearance'), async (req, res) => {
  try {
    const { envelopeId, documentId, recipientId, certificateId, signerName } =
req.body;
    const appearanceImageBuffer = req.file ? req.file.buffer : null;

    const resObj = await signAndEmbed({ envelopeId, documentId, recipientId,
certificateId, signerName, appearanceImageBuffer });
    res.json({ ok: true, ...resObj });
  } catch (err) {
    console.error('Sign error', err);
    res.status(500).json({ ok: false, error: err.message });
  }
});

module.exports = router;
```

Frontend helper (React) — minimal change to DocumentViewer

Place this inside your `DocumentViewer` component or a small helper. It sends the signature appearance image and signs.

```
// in DocumentViewer.jsx
async function handleSign(signatureFieldId, appearanceDataUrl) {
  // appearanceDataUrl is a base64 PNG data URL from canvas or signature pad
  const blob = await (await fetch(appearanceDataUrl)).blob();
  const form = new FormData();
  form.append('envelopeId', envelopeID);
  form.append('documentId', document.id);
  form.append('recipientId', currentUserId);
  form.append('certificateId', selectedCertificateId); // choose from UI
  form.append('signerName', currentUserName || 'Signer');
  form.append('appearance', blob, 'appearance.png');
  form.append('signatureFieldId', signatureFieldId);

  const response = await fetch('/api/e-sign/sign', {
    method: 'POST',
    body: form
  });
  const json = await response.json();
  if (!json.ok) throw new Error(json.error || 'Sign failed');
  return json;
}
```

Notes & required DB/model changes

1. `DigitalSignature` model should include fields: `pdfHash`, `cmsHash`, `certFingerprint`, `tsaToken`, `tsaTokenBytes` (bytes optional), `signedAt`.
2. Add unique index on `SignatureFields` for `{ documentId, recipientId, type }` to avoid race conditions.
3. `AuditTrail` entries are used liberally — ensure the model accepts `details` as a mixed object.

Migration & checklist before deploying to production

- Replace in-memory PEM usage with KMS/HSM-backed signing (preferred). The code above keeps PEM flow for compatibility.
- Ensure `saveSignedPdf` writes to a safe storage (S3 or equivalent) and returns the saved path. Do not expose local filesystem paths in public responses.

- Add tests: multi-signer sign order, signature validation after each sign, concurrency tests for simultaneous sign requests.
- Add monitoring/alerts for signing failures and TSA failures.

If you want, I can now: - produce the **KMS/HSM adapter** (AWS KMS / Azure Key Vault) so you can avoid storing private keys; - add a **worker/queue** implementation (Bull/bee-queue) for signing; or - write Jest/Mocha integration tests that verify signatures remain valid after subsequent signatures.

Tell me which of those you want next.